Computer Organization LAB

计算机 2405 邵凯伟

2026 年 10 月 26 日

目录

0 环境搭建 **2**

0.1 logisim 2

0.2 verilog 2

1 实验一 组合电路设计 **2**

1.1 logisim 实现 3

1.1.1 多路复用器 3

1.1.2 编码器 3

1.1.3 译码器 5

1.2 verilog 实现 5

1.2.1 多路复用器 5

1.2.2 优先编码器 7

1.2.3 译码器 7

1.3 verilog 仿真 8

1.3.1 优先编码器 8

1.3.2 译码器 9

0 环境搭建

0.1 logisim

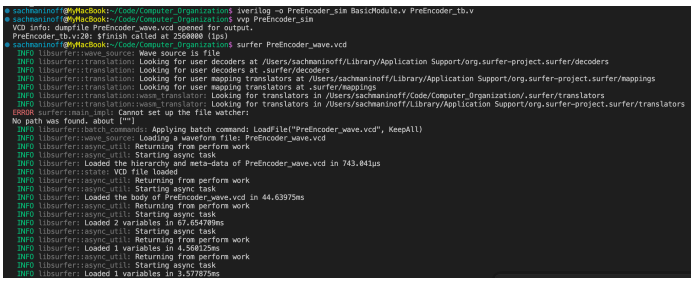
直接下载

0.2 verilog

采用 iverilog + surfer 方案 
使用`iverilog -o output code1.v code2.v`编译
使用`vvp output`模拟运行，（Verilog Virtual Processor）
使用`surfer filename.vcd`查看波形
surfer 原料: 在仿真文件中加入

```
initial begin
    $dumpfile("filenae.vcd");
    $dumpfile(0, modulename);
end
```

生成 vcd 文件
输出示例:



1 实验一 组合电路设计

基础门电路设计：多输入门电路, 复用器
基础功能模块设计：编码器, 译码器、、

1.1 logisim 实现

1.1.1 多路复用器

二路复用器

如图 1。

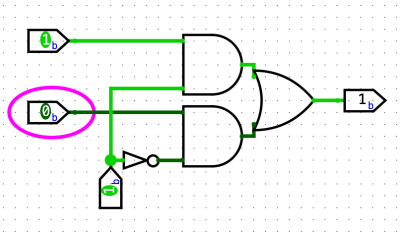


图 1: 二路复用器

三态门实现二路复用器

如图 2，用三态门代替与门，复用三态门线与省略或门。

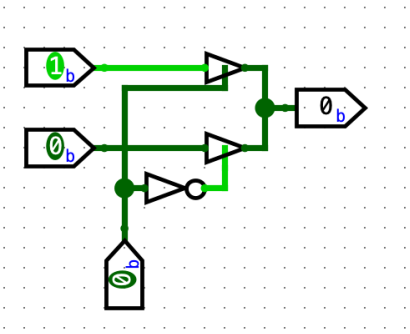


图 2: 三态门二路复用器

四路复用器

采用两层二路复用器实现，如图 3。

1.1.2 编码器

8-3 编码器

根据真值表，只需进行或逻辑:

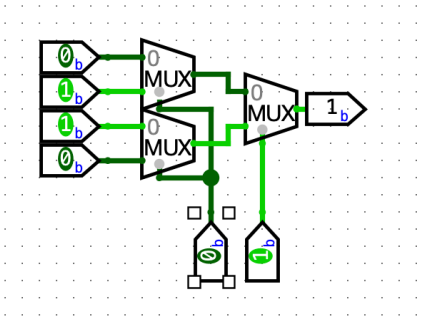


图 3: 四路复用器

$$\begin{cases} Y_2 = I_4 + I_5 + I_6 + I_7 \\ Y_1 = I_2 + I_3 + I_6 + I_7 \\ Y_0 = I_1 + I_3 + I_5 + I_7 \end{cases}$$

实现如图 4。

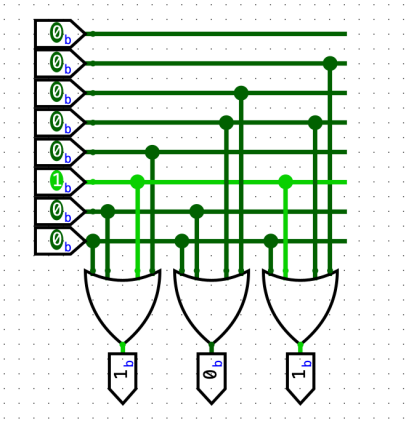


图 4: 8-3 编码器

8-3 优先编码器

逻辑分为两部分：仅点亮最高优先级的数据线，普通 8-3 编码器。

优先级部分：每一个输入 D_k 取决于 D_0 D_{k-1} 的非的与，再与上 I_k（图 5中的七个与门），非之与部分可变为或非门，（图中两个或非门用于简化低位逻辑），避免使用过多输入的与门。最后三个或门为普通 8-3 编码器。

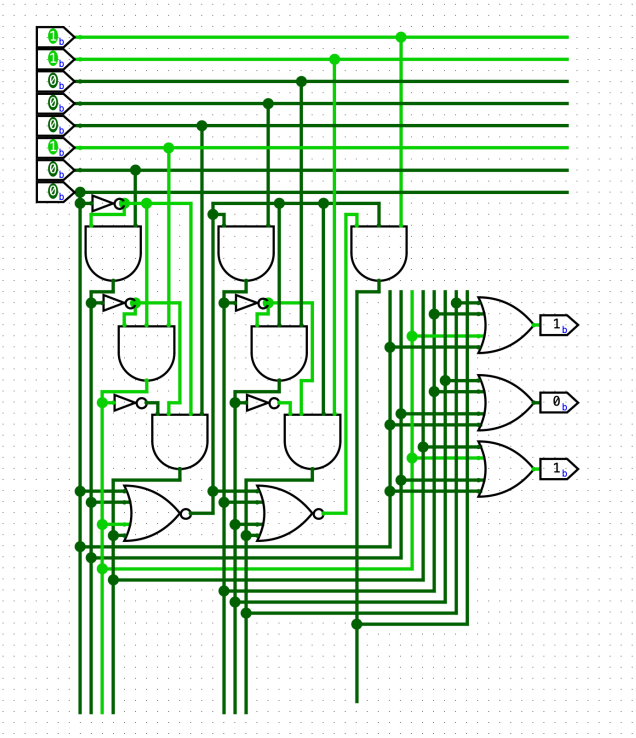


图 5: 8-3 优先编码器

1.1.3 译码器

3-8 译码器

如图 6, 使用与阵列即可。

1.2 verilog 实现

1.2.1 多路复用器

使用参数化路数量`#(parameter ways = 4)`，易于复用代码
`$clog2()`为内置函数，选择位宽为输入位宽的对数
最后将`data_in[sel]`作为输出即可。

```
module MUX #(
    parameter ways = 4
) (
    input wire [ways - 1 : 0] data_in,
```

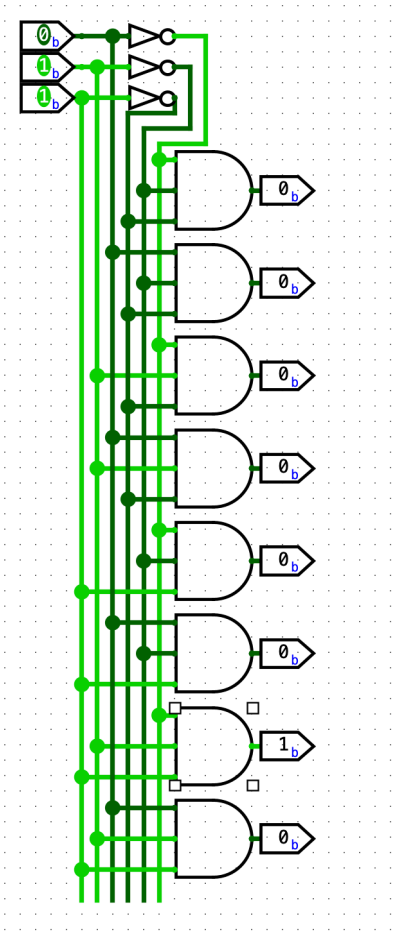


图 6: 3-8 译码器

```
    input wire [$clog2(ways) - 1 : 0] sel,
    output wire data_out
);
    assign data_out = data_in[sel];
endmodule
```

Listing 1: 多路复用器

1.2.2 优先编码器

输出在always块中赋值，将wire改为reg，用flag模拟break语句。
在for循环外先给data_out赋初值，以防止data_in全为 0 时data_out变为寄存器.

```
module PreEncoder #(
    parameter ways = 8
) (
    input wire [ways - 1 : 0] data_in,
    output reg [$clog2(ways) - 1 : 0] data_out
);
    reg flag;
    integer i;
    always @(*) begin
        flag = 0;
        i = 0;
        data_out = 0;
        for (i = ways - 1; i >= 0; i = i - 1)  begin
            if (!flag) begin
                if (data_in[i]) begin
                    flag = 1;
                    data_out = i;
                end
            end
        end
    end
endmodule
```

Listing 2: 优先编码器

1.2.3 译码器

data_out的第data_in位为 1，其余均为 0。
故data_out = 1 << data_in。

```
module Decoder #(
    parameter ways = 8
) (
    input wire [$clog2(ways) - 1 : 0] data_in,
```

```
    output wire [ways - 1 : 0] data_out
);
    assign data_out = 1'b1 << data_in;
endmodule
```

Listing 3: 译码器

1.3 verilog 仿真

1.3.1 优先编码器

```
`timescale 1ns / 1ps
module PreEncoder_tb;
    parameter ways = 8;
    reg [ways - 1 : 0] data_in;
    wire [$clog2(ways) - 1 : 0] data_out;

    integer i;

    PreEncoder #(
        .ways(ways)
    ) PreEncoder_sim (
        .data_in(data_in),
        .data_out(data_out)
    );
    initial begin
        for (i = 0; i < (1 << ways); i = i + 1) begin
            data_in = i;
            #10;
        end
        $finish;
    end
    initial begin
        $dumpfile("PreEncoder_wave.vcd");
        $dumpvars(0, PreEncoder_tb);
    end
endmodule
```

Listing 4: 优先编码器仿真

波形图

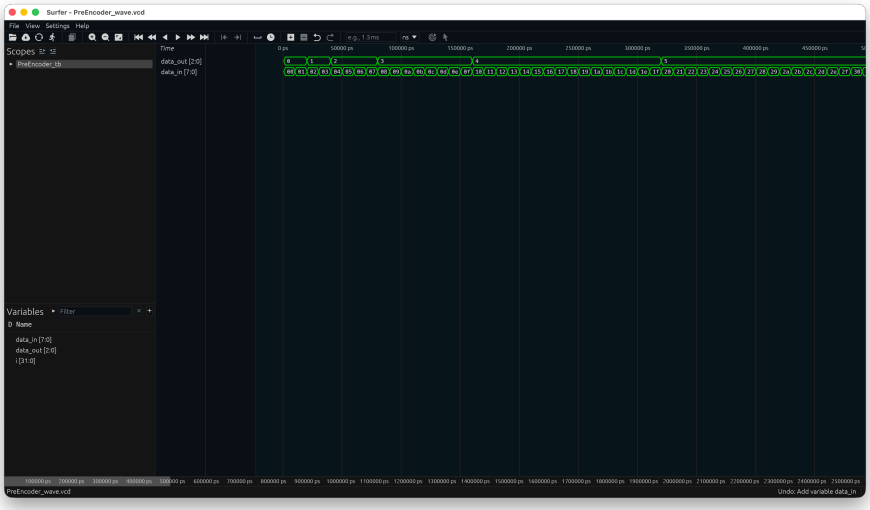


图 7: 优先编码器仿真波形

1.3.2 译码器

```
`timescale 1ns / 1ps
module Decoder_tb;
    parameter ways = 8;
    reg [$clog2(ways) - 1 : 0] data_in;
    wire [ways - 1 : 0] data_out;

    integer i;

    Decoder #(
        .ways(ways)
    ) Decoder_sim (
        .data_in(data_in),
        .data_out(data_out)
    );
    initial begin
        for (i = 0; i < ways; i = i + 1) begin
            data_in = i;
```

```
18              #10;
19          end
20          $finish;
21      end
22      initial begin
23          $dumpfile("Decoder_wave.vcd");
24          $dumpvars(0, Decoder_tb);
25      end
26  endmodule
```

Listing 5: 编码器仿真

波形图

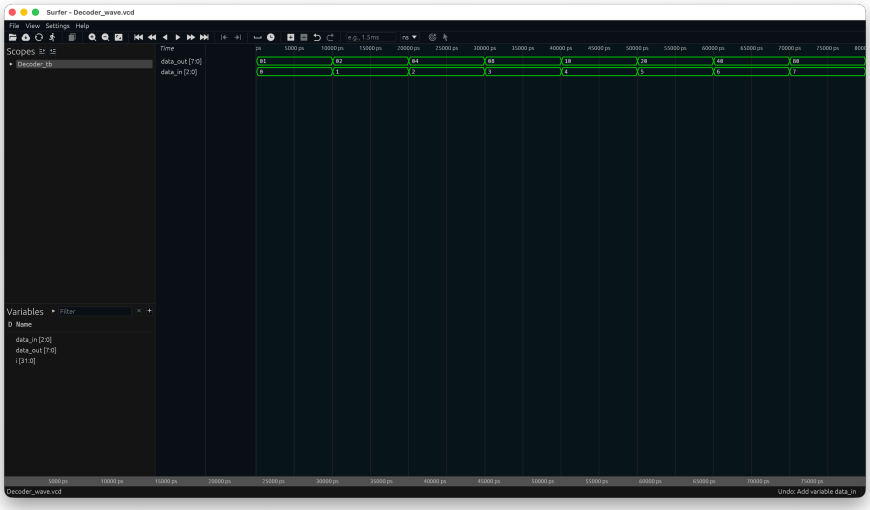


图 8: 译码器仿真波形